

Some important indicative examples are given below.

Email headers:

```
From: "Fred Wilson" <Fred@usv.com>
Reply-To: "Fred Wilson" <Fred@usv.com>
Sender: FeedBlitz <feedblitz@mail.feedblitz.com>
Return-Path: 39646264_304_4941999_fbz@mail.feedblitz.com
To: Saifi <saifi@containerlogic.co>
Date: Thu, 07 May 2015 07:14:54 -0400
MIME-Version: 1.0
Content-Type: multipart/alternative;
    charset="UTF-8";
    boundary="f33dBLItz_mIME_pART_BOUNDARY==39646264==v1_"
Message-ID: <MAIL045PyXMBJtZ2Ib007e3aa8@Mail04.feedblitz.com>
X-OriginalArrivalTime: 07 May 2015 11:14:54.0345 (UTC)
FILETIME=[0B48AF90:01D088B7]
```

X.509 certificate fragment:

```
<element name="X509Data" type="ds:X509DataType"/>
<complexType name="X509DataType">
  <sequence maxOccurs="unbounded">
    <choice>
      <element name="X509IssuerSerial" type="ds:X509IssuerSerialType"/>
      <element name="X509SKI" type="base64Binary"/>
      <element name="X509SubjectName" type="string"/>
      <element name="X509Certificate" type="base64Binary"/>
      <element name="X509CRL" type="base64Binary"/>
      <any namespace="*" other=" processContents="lax"/>
    </choice>
  </sequence>
</complexType>
<complexType name="X509IssuerSerialType">
  <sequence>
    <element name="X509IssuerName" type="string"/>
    <element name="X509SerialNumber" type="integer"/>
  </sequence>
</complexType>
```

JSON dump of Docker container:

```
{[
  "Architecture": "amd64",
  "Author": "Saifi Khan saifi@containerlogic.co",
  "Comment": "",
  "Config": {
    ..
    "Entrypoint": [
      "startdb.sh"
    ],
    "Env": [
      "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/
```

```
usr/bin:/sbin:/bin"
  ],
  "Hostname": "5bf0de3d0926",
  "Image": "d9065eab448785cb11377",
  "f381135f1e345d31883ae39bb646e9129ea9b3b05b3",
  "Memory": 0,
  "MemorySwap": 0,
  "NetworkDisabled": false,
  ..
  "Volumes": null,
  "WorkingDir": ""
},
"Container": "10dee820e7e5dced36523ed7ab
7f09aaf8d177b657985514c120f7a1a605e96b7",
"ContainerConfig": {
  ..
  },
  "Hostname": "5bf0de3d0926",
  "Image": "d9065eab448785cb11377f3811
35f1e345d31883ae39bb646e9129ea9b3b05b3",
  ..
  },
  "Created": "2015-05-05T17:55:02.969770577Z",
  "DockerVersion": "1.3.2-dev",
  "Id": "2d37849cf381b7023f067b4
dd4445d21981e43d9fced1ed345ff614d1170a65e",
  "Os": "linux",
  "Parent": "d9065eab448785cb11377f
381135f1e345d31883ae39bb 646e9129ea9b3b05b3",
  "Size": 0,
  "VirtualSize": 17513894
}]
```

As seen in the examples above, header fields, JSON or XML entities belonging to the same class type may have different attributes even though they are grouped together.

Two important points here are:

- Semi-structured data usually has a self-describing structure
 - The ordering constraint is not important
- This is in contrast to the formal relational structure that is enforced by RDBMS and data tables.

With the preponderance of JavaScript in Web-centric applications and services, attribute key pairs are expressed using JSON and consumed using REST endpoints.

Thus key-value data stores can use the JSON format to natively store data. CouchBase and MongoDB take this approach.

Schema-less databases

The rallying cry of the MongoDB evangelist is that it is a schema-less database. This needs to be understood in the context of the database and the in-memory data structure. First of all, there is *still an implicit inferred* schema. The database also supports non-uniform data types, for example, relational schema for the student table.